Training a small GPT on TinyStories in one hour: model size against the tokens it can see

Group Composition: November 2026

1. Kerem Ozemre, A0350378Y
2. Tonio Kowalke (Kowalke Jeri Tonio Janek), A0350429E
3. Steffany Lara (Lara Muy Steffany Mishell), A0350565H
4. Ahmed Haj Hassine (Haj Hassine Ahmed Ben Abdellaziz), A0342398U

Github Repository: github.com/KeremOzemre/Optimization-for-Large-Scale-Data

Abstract

We train GPT models from random weights on TinyStories. Each run gets one hour on the same GPU slice. We first compare five model sizes (16M to 253M parameters) and four fractions of the training data, 20 runs in total. The 124M model (XL) has the lowest loss. The 253M model (XXL) is not better, although it has twice the parameters, because in one hour it processes only half as many tokens. The first comparison had one seed and unequal validation samples, so we checked it twice: we scored the saved checkpoints again on the complete validation set, and we trained XL and XXL again with three seeds each. XL wins in all three pairs (mean cross-entropy 1.4947 against 1.5244, about 2.9% lower perplexity). The amount of training data changes the loss by about 0.01 nats or less. The validation-selected model reaches a test perplexity of 4.41 (0.524 bits per byte). A matched-token control settles why XXL loses: trained on the same 143M tokens with a step-based schedule, XXL beats XL by 0.053 nats, so the one-hour ranking comes from throughput, not capacity. Halving XXL's learning rate did not close the one-hour gap.

1 Budget, initialisation and evaluation objective

Budget. We fixed the budget before the final runs: 3600 seconds of wall-clock time inside the training loop for each run, on one NVIDIA H100 NVL MIG slice (`3g.47gb`), in `bfloat16`. The timer includes the periodic validation and logging. Data preparation, model creation, the final validation and saving the checkpoint are outside the timer. No run was resumed. The 20 runs of the first comparison and the 6 repeated runs use 26 slice-hours of one-hour training. The matched-token pair (Section 4.3) uses a step budget of 35,000 updates instead

of the clock and, with the reduced-rate one-hour run, adds about 2.5 slice-hours on the same MIG slice. Four extra runs (Section 4.4) add 2.5 hours on a GPU with a different label, and we treat them separately. Pilot runs and all offline scoring are not in the budget.

Initialisation. All models start from random weights with the nanoGPT initialisation [4]. We do not use pretrained weights or embeddings. The only external component is the GPT-2 tokenizer, which the project rules allow.

Evaluation objective. We measure cross-entropy (CE, nats per token) on held-out TinyStories text. It is the quantity the model is trained on, so it reads language-modelling quality directly. We also report perplexity, exp(CE), and bits per byte for the test set, because per-token numbers cannot be compared with groups that use another tokenizer. These metrics say how well the model predicts the next token. They do not say if a generated story is coherent, so we also read samples (Section 4.5), without scoring them.

2 Research question and background

Our question is practical: *with one hour on a fixed GPU, which model size predicts held-out text best, and does it matter how much training data is available?*

A larger model can fit more patterns, but every update costs more. With a fixed clock, more capacity means fewer updates and fewer tokens seen. Hoffmann et al. [3] study this trade-off for a fixed amount of compute and find that model size and training tokens should grow together. Their result is for much larger models and data, and their budget is FLOPs, not time. So we cannot copy their ratio, and we measure the balance for our setting. For the data axis, the reference is Muennighoff et al. [6]: they report that repeating data for up to about four epochs costs almost nothing. Our small models repeat the 25% subset about eight times, so we go a bit further than their range.

We chose TinyStories [1] because it was built for small models: models with a few million parameters already write simple, mostly correct stories. Differences between our configurations should then be visible in one hour.

3 Experimental setup

Data. `roneneldan/TinyStories` from Hugging Face. The training split has 2,119,719 stories (473,992,236 tokens). We split the official validation set 80/20 at story level with seed 42, which gives 17,592 validation stories (3,808,644 tokens) and 4,398 test stories (957,274 tokens). The 25/50/75% subsets have about 118, 237 and 355 million tokens. We did not check the dataset for duplicated stories.

Tokenizer. GPT-2 BPE from `tiktoken` (50,257 entries), with an end-of-text token after each story. It is the same for all runs.

Model. The nanoGPT decoder-only Transformer [4, 7]: causal attention, learned position embeddings, tied input and output embeddings, context length 256, dropout 0.1.

Table 1 lists the five sizes. Depth, width and number of heads change together, so we cannot separate their effects.

Table 1: The five model sizes. The effective batch is always 16 sequences (4096 tokens) per update. The last three columns are for the 100% data runs of the first comparison.

Size	Layers	Width	Heads	Params (M)	Micro × accum.	Tokens/s (k)	Tokens (M)
Small	4	256	4	16.08	16×1	273.4	984.3
Medium	6	384	6	30.02	16×1	204.1	734.6
Large	8	512	8	51.04	16×1	156.2	562.4
XL	12	768	12	123.75	8×2	77.7	279.8
XXL	16	1024	16	253.09	4×4	39.9	143.7

Training recipe. AdamW [5] with peak learning rate 3×10^{-4}, weight decay 0.1, betas (0.9, 0.95) and gradient clipping at 1.0. The schedule follows the clock and not the step count: a linear warmup during the first 2% of the hour (72 seconds), then cosine decay to 10% of the peak at the end. We did this because the budget is in time. A fast model and a slow model then finish at the same point of the schedule, and no run is cut in the middle of its decay, which would make it look worse than it is [2]. Training windows are sampled with replacement from the token stream. The recipe is the same for all sizes. We did not tune the learning rate for each size, and we come back to this in Section 5.

Validation and test. During training we score a fixed sample of validation windows. The checkpoint at the end of the hour is the one we keep. We do not pick an earlier one. Model choices use validation only. The test set is scored offline with non-overlapping 256-token windows (957,184 tokens, the short tail is dropped).

Sequence of experiments. (1) A 5×4 comparison: five sizes, four data fractions, seed 42. (2) Because XL and XXL were very close, a check of the measurement: four saved checkpoints scored again on the complete validation set. (3) A check of the training noise: XL and XXL trained again on the full data with seeds 42, 43 and 44. (4) A matched-token control: XL and XXL from fresh weights for the same 35,000 updates (143M tokens) with a step-based schedule, to separate throughput from capacity. (5) A reduced learning-rate run for XXL, and four exploratory runs with a shorter budget or a longer context. Steps 2 to 5 were decided after we saw step 1, so they are targeted follow-ups and not independent experiments.

4 Results

4.1 Model size and data fraction (20 runs)

Figure 1 shows the first comparison, and Table 4 in the appendix has all 20 runs. With the full data, the validation CE falls from 1.839 (Small) to 1.644 (Medium), 1.546 (Large) and 1.507 (XL). Then it stops: XXL has 1.511. The step from Large to XL is worth 0.039 nats. The step from XL to XXL is worth nothing, or slightly less than nothing.

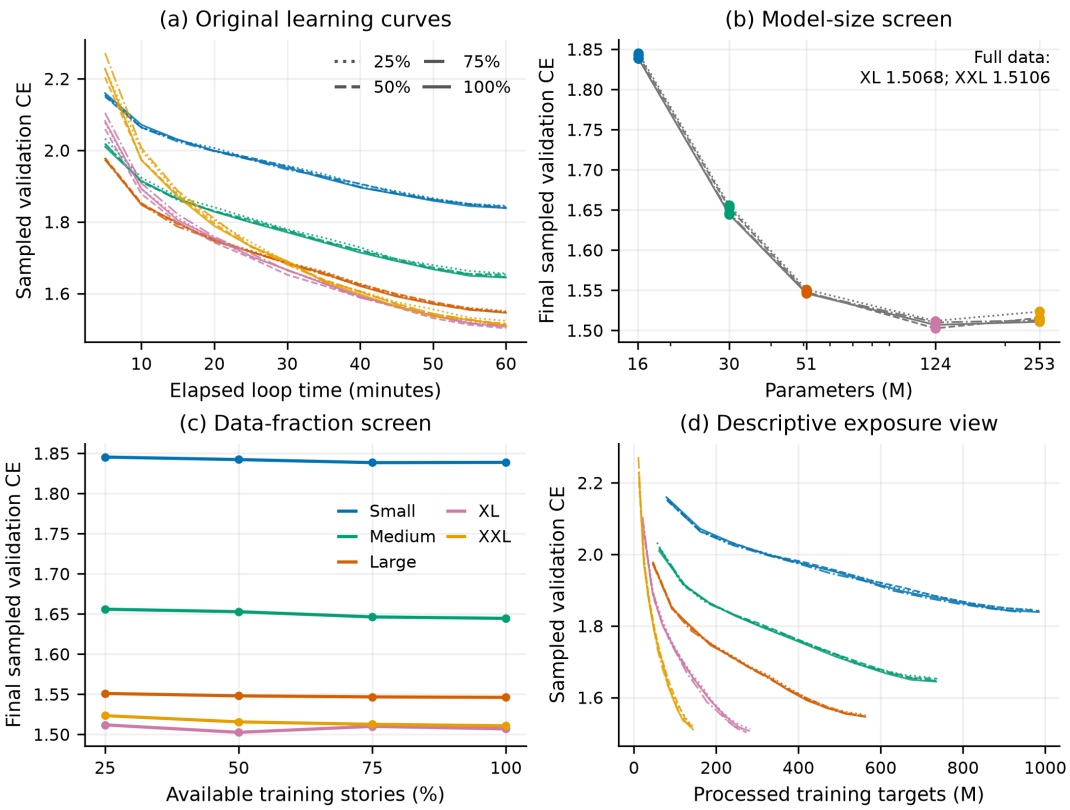


Figure 1: The 20-run comparison. (a) Validation CE against time. (b) Final CE against model size. (c) Final CE against data fraction. (d) The same curves against processed tokens. Colours are sizes, line styles are data fractions.

The data fraction matters much less. Inside each size, 100% of the data is better than 25% by 0.005 to 0.013 nats, and the fractions in the middle are not in a clear order (panel c). So more data seems to help a little, but with one seed per cell we cannot separate this from noise. What we can say is that using only a quarter of TinyStories costs at most about 0.01 nats in this setting.

4.2 How many tokens each model sees

The reason for the flat end of panel (b) is in Table 1. In the same hour Small processes 984M tokens and XXL only 144M. The throughput falls by a factor of 6.9 from Small to XXL. In units of the full training set (474M tokens), Small sees it 2.08 times, Medium 1.55, Large 1.19, XL 0.59 and XXL 0.30 times. XXL would need about 3 hours and 18 minutes to see the full data once.

This also corrects an explanation we had in an early draft, that the data fraction does not matter because no run finishes one pass. That is wrong for most of the grid: 15 of the 20 runs process more tokens than their training subset has, and Small with 25% of the data goes through its subset 8.2 times. Its loss is still only 0.007 nats worse than Small with

all the data. This agrees with Muennighoff et al. [6], and in our case it holds up to eight repetitions. A note of care: we sample windows with replacement, so "8.2 times" counts processed tokens, it does not mean every story was seen eight times. We also checked that the token counters in all 262,432 log rows are equal to steps $\times$ 4096.

4.3 Is XL really better than XXL?

The gap between XL and XXL in the first comparison is 0.004 nats, with one seed. There was a second problem, which we found late. The validation sample during training was 16 microbatches, and the microbatch size depends on the model. So Small, Medium and Large were scored on 65,536 tokens, XL on 32,768 and XXL on 16,384. A gap of 0.004 measured like this means very little. We did two checks.

Same validation data. We scored four saved checkpoints on the complete validation set (3,808,643 tokens each). Table 2 shows that the gap between XL and XXL grows to 0.034 nats, and XL stays ahead. The order of the two XL runs changes: with the complete set, 100% of the data is 0.002 better than 50%, and before it was the opposite. So the original choice of XL with 50% data was inside the noise of a small sample.

Table 2: Saved checkpoints scored on the complete validation set, compared with the sampled validation used during training.

Checkpoint	Sampled CE	Complete CE	Complete PPL
Large, 100%	1.5460	1.5469	4.697
XL, 50%	1.5025	1.4911	4.442
XL, 100%	1.5068	1.4893	4.434
XXL, 100%	1.5106	1.5236	4.589

Three seeds. We trained XL and XXL again on the full data with seeds 42, 43 and 44, with the same recipe and the same GPU label, and scored the final checkpoints on the complete validation set (Table 3, Figure 2). XL is better in every pair, and every XL run is better than every XXL run. The mean CE is 1.4947 for XL (standard deviation 0.0038) and 1.5244 for XXL (0.0098). The mean paired gap is 0.030 nats, which is about 2.9% in perplexity (4.458 against 4.592).

We do not want to oversell this. With three pairs, a paired t interval at 95% is $[-0.002,\ 0.061]$, so it includes zero, and three points cannot tell us if the normal assumption is reasonable. What we have is the same direction three times and a gap about eight times larger than the spread between XL seeds. We think this is enough to say that XL is the better choice for one hour with this recipe. It is not a significance test.

One detail in Table 3 is useful. The seed-44 XXL run processed 9.8% more tokens than the seed-43 run, on the same GPU label, and it is also the best XXL run. So the spread between XXL runs is not only the random seed. It is also how fast the shared GPU was during that hour. For a slow model this matters more.

Table 3: Repeated one-hour runs on the full data. CE on the complete validation set. A positive gap means XL is better.

Seed	XL CE	XXL CE	Gap (XXL−XL)	XL tokens (M)	XXL tokens (M)
42	1.4904	1.5298	0.0394	271.0	138.1
43	1.4960	1.5303	0.0343	267.9	133.3
44	1.4976	1.5131	0.0155	267.9	146.4

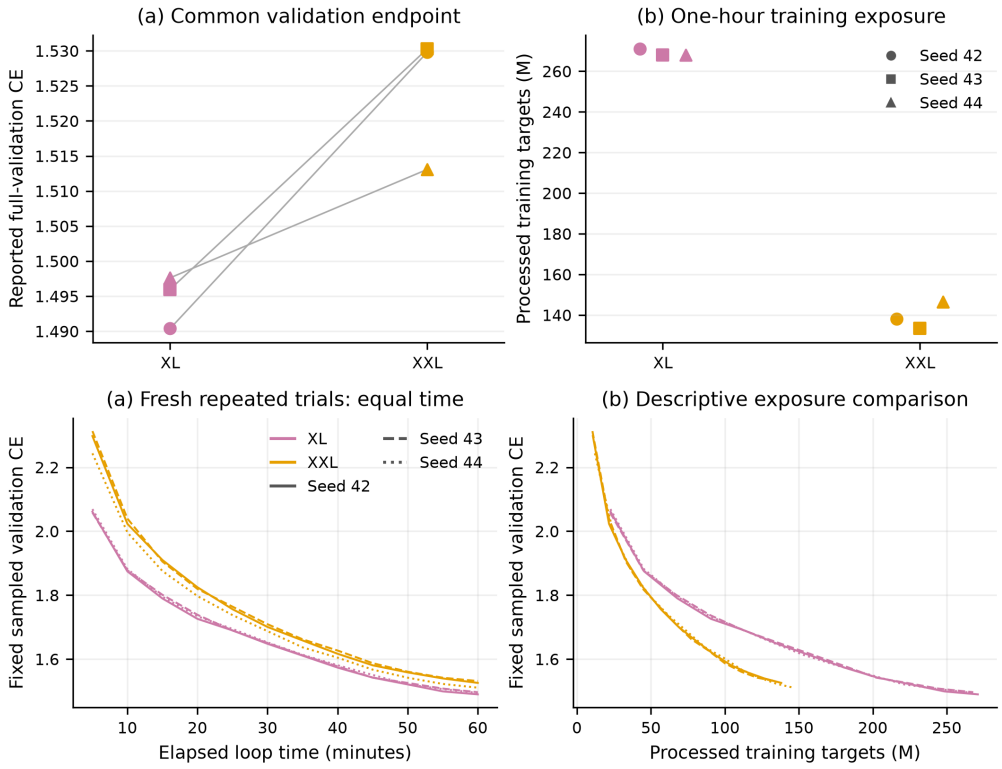


Figure 2: Repeated runs. Top: final CE on the complete validation set and processed tokens, by seed. Bottom: sampled validation CE against time (left) and against processed tokens (right).

Why XXL loses. The bottom-right panel of Figure 2 gives the most interesting reading. At the same number of processed tokens, XXL is ahead: at 120M tokens its CE is about 1.55 and XL is at about 1.68. XXL learns more from each token, as expected for a larger model. It loses because XL gets 1.93 times as many tokens in the hour. All six curves are still falling in the last five minutes, so no model has converged. This suggests XXL is limited by time and not by capacity. One reading of the curves is confounded, though: the schedule follows the clock, so at 120M tokens XL is still at a high learning rate and XXL is near the end of its decay, and part of the gap in that panel can come from this.

Matched-token control. To remove this confound we trained XL and XXL from fresh weights for the same 35,000 updates (143,360,000 tokens each), with a step-based cosine schedule, the same peak learning rate, seed 42 and the same MIG slice. At equal tokens

the ranking flips: XXL reaches complete-validation CE 1.521 against 1.574 for XL, a gap of 0.053 nats (5.2% in perplexity) in XXL's favour. XXL needed 62 minutes for these tokens and XL only 31. So, with the clock confound removed, the extra capacity of XXL does help per token, and it loses in one hour only because it processes 1.93 times fewer tokens. This is one seeded pair, so we weight the direction more than the exact size of the gap.

4.4 Shorter budget and longer context (exploratory)

We ran four more runs with seed 42 and the full data. With a 30-minute budget, Large has the lowest validation CE (1.580), then XL (1.633) and XXL (1.680). With one hour XL is first. So the best size moves up when the budget grows, which is what the token argument predicts. In the fourth run, XL with context 512 reaches 1.456 against 1.507 for context 256, and it processes fewer tokens (190M against 280M).

We report these as leads and not as results. The four runs were recorded on a GPU labelled `H100 NVL`, without the MIG slice, and the throughput is not consistent with the main runs (Large was faster than before, XL slower). The context-512 run also doubles the tokens per update, from 4096 to 8192. With these differences we cannot say how much of each change comes from the budget or from the context.

4.5 Test results and generated stories

The model selected on validation in the first comparison is XL with 50% of the data. Its test CE is 1.4846, the perplexity is 4.41 and it has 0.524 bits per byte. The test results of all 20 runs are in Table 4, and the order of the sizes is the same as on validation: Small 6.26, Medium 5.16, Large 4.66, XL 4.40 and XXL 4.56 in perplexity for the full-data runs. We have to be clear about one thing. We scored all 20 checkpoints on the test set at the same time, so the test set was seen before the follow-up decisions. For this reason the repeated runs are compared on validation only, and we have no test scores for them.

We also generated five stories per model with fixed prompts (temperature 0.8, top-k 40, 150 new tokens). All models write fluent text in the TinyStories style, and all of them make errors that the loss does not show. For the prompt "Tom had a big red ball.", Small changes the name to Tim, XL has a girl who says to Tom "I want to pet you", and in the XXL story Sam says "I'm Sam. I'm Tom." Five prompts with one sample each cannot rank the models, so we do not claim that the lower loss of XL gives better stories.

5 Discussion and limitations

The first comparison, the scoring on the complete validation set and the repeated runs agree: for one hour on this GPU slice with our recipe, the 124M model is the best of the five. The main trade-off is throughput against capacity. A bigger model uses each token better but gets fewer of them, and at one hour the balance is near 124M parameters. The matched-token control makes this explicit: at equal tokens the larger model is ahead, so the one-hour

result is about time, not a ceiling on what XXL can learn. The 30-minute runs point the same way, with the balance at a smaller model.

The limits are these. First, we used one learning rate for all sizes. Larger models often need a smaller one, so we also trained XXL for one hour with half the peak rate (1.5×10^{-4}); it did not help, with complete-validation CE rising to 1.546 from 1.530 at the default. One value is not a search, but the shared recipe is not an obvious handicap here. Second, three seeds are few, and the finalists were chosen after we saw the first results. Third, we measure wall-clock time on a shared GPU slice, and the speed changed by up to 10% between runs of the same model. Fourth, the data fraction results have one seed and differences of 0.01 nats, so we can only say the effect is small. Fifth, the sampled validation during training used fewer tokens for the bigger models. We only found this after the first comparison, and it is the reason why we scored the checkpoints again.

About reproducibility: configurations, logs and score tables for every run are in the repository, with the follow-ups under `configs/followups/` and `analysis/followups/`. All checkpoints are on Hugging Face: the 20 from the first comparison, the 6 repeated-seed runs, and the matched-token and reduced-rate runs, each with its training metrics and run configuration. Every reported number can be recomputed from a released checkpoint and the evaluation code in the repository.

The experiments we would do next build on the matched-token control: repeat it over several seeds, and give XXL a proper per-size learning-rate search instead of the single reduced rate we tried. A longer wall-clock budget would then show whether XXL overtakes XL on the clock, which our token accounting predicts but does not prove.

6 Conclusion

With one hour per run, the best model is not the biggest one. XL (124M) beats XXL (253M) by about 0.03 nats in three repeated pairs, because it processes almost twice as many tokens. A matched-token control confirms the mechanism: given the same 143M tokens, XXL is ahead by 0.053 nats, so the one-hour ranking is set by throughput, not capacity. The amount of training data changes little, even when the small models repeat a quarter of the data eight times. The selected model has a test perplexity of 4.41. What stays open is whether a longer budget or a per-size learning rate lets XXL win on the clock, and whether context 512 is really better: halving XXL's rate for one hour did not help, and the context result is from a single run on a different GPU label.

References

[1] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023. URL https://arxiv.org/abs/2305.07759.

[2] Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro Von Werra,

and Martin Jaggi. Scaling laws and compute-optimal training beyond fixed training durations. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://arxiv.org/abs/2405.18392.

[3] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. URL https://arxiv.org/abs/2203.15556.

[4] Andrej Karpathy. nanoGPT. Software repository, n.d. URL https://github.com/karpathy/nanoGPT/tree/3adf61e154c3fe3fca428ad6bc3818b27a3b8291. MIT license. Source commit 3adf61e154c3fe3fca428ad6bc3818b27a3b8291.

[5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

[6] Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2305.16264.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1706.03762.

A All 20 runs and how to reproduce them

The commands are in `README.md`: `run.py prepare`, `run.py train` and `run.py evaluate`, each with a file from `configs/`. `eval_hf.py` scores the saved checkpoints on the test set and `analyze.py` builds the tables and figures. Per-run logs are in `results/` and `runs/`; the follow-up configs are in `configs/followups/` and the score tables of Section 4.3 are in `analysis/followups/` (seed, matched-token and learning-rate subfolders).

Table 4: The 20-run comparison (seed 42, one hour). "Passes" is processed tokens divided by the size of the training subset. Validation is the sampled one, test is the full test set (957,184 tokens).

Size	Data (%)	Tokens (M)	Passes	Val CE	Test CE	Test PPL	Test bpb
Small	25	972.0	8.21	1.8452	1.8422	6.311	0.650
Small	50	985.5	4.16	1.8421	1.8396	6.294	0.649
Small	75	940.8	2.65	1.8383	1.8378	6.283	0.648
Small	100	984.3	2.08	1.8386	1.8340	6.259	0.647
Medium	25	719.1	6.07	1.6558	1.6502	5.208	0.582
Medium	50	737.3	3.11	1.6527	1.6485	5.199	0.582
Medium	75	738.3	2.08	1.6462	1.6430	5.171	0.580
Medium	100	734.6	1.55	1.6444	1.6415	5.163	0.579
Large	25	563.2	4.75	1.5509	1.5475	4.700	0.546
Large	50	557.9	2.35	1.5479	1.5445	4.686	0.545
Large	75	562.0	1.58	1.5466	1.5408	4.668	0.544
Large	100	562.4	1.19	1.5460	1.5390	4.660	0.543
XL	25	283.3	2.39	1.5118	1.4918	4.445	0.526
XL	50	273.9	1.16	1.5025	1.4846	4.413	0.524
XL	75	269.1	0.76	1.5099	1.4890	4.433	0.525
XL	100	279.8	0.59	1.5068	1.4824	4.403	0.523
XXL	25	143.2	1.21	1.5233	1.5291	4.614	0.539
XXL	50	148.2	0.63	1.5155	1.5182	4.564	0.536
XXL	75	141.1	0.40	1.5125	1.5231	4.587	0.537
XXL	100	143.7	0.30	1.5106	1.5174	4.560	0.535